

Q2Static/Dynamic Link Library A math tutoring app needs a reusable complex number library. Build a static library libcomplex.a with functions for addition and multiplication of complex numbers. Write a test program that uses the library. Write complex.c with add_complex() and mul_complex() functions. Compile to object file and archive using ar rcs. Link test program with gcc test.c -L. -lcomplex -o test and run

complex.h

```
void add_complex(int a, int b, int c, int d);  
void mul_complex(int a, int b, int c, int d);
```

complex.c

```
#include <stdio.h>  
  
#include "complex.h"  
  
// Addition  
void add_complex(int a, int b, int c, int d)  
{  
    int real;  
    int imag;  
    real = a + c;  
    imag = b + d;  
    printf("\nAddition = %d + %di\n", real, imag);  
}  
  
// Multiplication  
void mul_complex(int a, int b, int c, int d)  
{  
    int real;  
    int imag;  
    real = (a*c) - (b*d);  
    imag = (a*d) + (b*c);  
    printf("Multiplication = %d + %di\n", real, imag);  
}
```

test.c

```
#include <stdio.h>
#include "complex.h"
int main()
{
    int a, b, c, d;
    printf("Enter first complex number:\n");
    printf("Real Part: ");
    scanf("%d", &a);
    printf("Imaginary Part: ");
    scanf("%d", &b);
    printf("\nEnter second complex number:\n");
    printf("Real Part: ");
    scanf("%d", &c);
    printf("Imaginary Part: ");
    scanf("%d", &d);
    add_complex(a, b, c, d);
    mul_complex(a, b, c, d);
    return 0;
}
```

Link

// Static Link

1. gcc -c geometry.c -o geometry.o
2. ar rcs libgeometry.a geometry.o
3. gcc main.c -L. -lgeometry -lm -o staticprog
4. ./staticprog

// Dynamic Link

1. gcc -c -fPIC geometry.c -o geometry.o
2. gcc -shared -o libgeometry.so geometry.o -lm
3. export LD_LIBRARY_PATH=\$PWD:\$LD_LIBRARY_PATH
4. gcc main.c -L. -lgeometry -lm -o dynamicprog
5. ./dynamicprog

Q3System Calls A file backup tool needs to copy a file from one location to another using only system calls (no standard C file functions). Simulate this for a file called source.txt. Use open(), read(), write(), and close() system calls only. Handle errors for each system call and print appropriate messages. Verify copy was successful using stat() to compare file sizes

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

int main()
{
    int source_fd;
    int dest_fd;
    char buffer[100];
    int bytes;
    struct stat source_stat;
    struct stat dest_stat;
    // Open source file
    source_fd = open("source.txt", O_RDONLY);
    if(source_fd < 0)
    {
        printf("Error opening source file\n");
        return 1;
    }
    printf("Source file opened successfully\n");
    // Create destination file
    dest_fd = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC,
0666);
    if(dest_fd < 0)
    {
        printf("Error creating destination file\n");
        close(source_fd);
        return 1;
    }
    printf("Destination file created successfully\n");
    // Copy data
    while((bytes = read(source_fd, buffer, sizeof(buffer))) > 0)
    {
        if(write(dest_fd, buffer, bytes) != bytes)
        {
```

```

        printf("Write error\n");
        close(source_fd);
        close(dest_fd);
        return 1;
    }
}

if(bytes < 0)
{
    printf("Read error\n");
    close(source_fd);
    close(dest_fd);
    return 1;
}

// Close files
close(source_fd);
close(dest_fd);
printf("File copied successfully\n");
// Compare file sizes using stat()
stat("source.txt", &source_stat);
stat("destination.txt", &dest_stat);
printf("\nSource File Size = %ld bytes\n", source_stat.st_size);
printf("Destination File Size = %ld bytes\n", dest_stat.st_size);
if(source_stat.st_size == dest_stat.st_size)
{
    printf("\nCopy Verified Successfully\n");
}
else
{
    printf("\nCopy Verification Failed\n");
}
return 0;
}

// Create a nano file and write some text in it before running the
program. For example:
// echo "Hello World!" > source.txt

```

Q4Multithreading A bank wants to calculate the sum of a large array of transactions. To speed things up, divide the array into 4 equal parts and compute the sum of each part in a separate thread, then combine results. Create an array of 100 integers,

spawn 4 threads using pthread_create() Each thread computes sum of its 25-element portion Use pthread_join() to collect results and print total

```
#include <stdio.h>
#include <pthread.h>
int arr[100];
int partial_sum[4];
void* calculate_sum(void* arg)
{
    int id = *(int*)arg;
    int start = id * 25;
    int end = start + 25;
    partial_sum[id] = 0;
    for(int i = start; i < end; i++)
    {
        partial_sum[id] =
            partial_sum[id] +
            arr[i];
    }
    printf("Thread %d Sum = %d\n", id, partial_sum[id]);
    return NULL;
}
int main()
{
    pthread_t threads[4];
    int id[4] = {0, 1, 2, 3};
    int total = 0;
    // Initialize array
    for(int i = 0; i < 100; i++)
    {
        arr[i] = i + 1;
    }
    // Create threads
    for(int i = 0; i < 4; i++)
    {
        pthread_create(&threads[i], NULL, calculate_sum, &id[i]);
    }
    // Wait for threads
    for(int i = 0; i < 4; i++)
    {
        pthread_join(threads[i], NULL);
    }
    // Combine sums
```

```

    for(int i = 0; i < 4; i++)
    {
        total = total + partial_sum[i];
    }
    printf("\nTotal Sum = %d\n", total);
    return 0;
}

```

Q5CPU Scheduling A hospital's triage system assigns CPU time based on arrival and priority. Given 5 patients (processes) with known arrival times and burst times, simulate FCFS scheduling and calculate average waiting time. Input: P1(0,6), P2(1,4), P3(2,2), P4(3,3), P5(4,1) — (arrival, burst) Implement FCFS in C, compute waiting time and turnaround time for each process Display a Gantt chart using dashes and print average waiting time

```

#include <stdio.h>
int main()
{
    int at[5] = {0, 1, 2, 3, 4};
    int bt[5] = {6, 4, 2, 3, 1};
    int wt[5];
    int tat[5];
    int ct[5];
    float avg_wt = 0;
    // Completion time
    ct[0] = at[0] + bt[0];
    for(int i = 1; i < 5; i++)
    {
        if(ct[i-1] < at[i])
        {
            ct[i] = at[i] + bt[i];
        }
        else
        {
            ct[i] = ct[i-1] + bt[i];
        }
    }
    // Turnaround time
    for(int i = 0; i < 5; i++)
    {
        tat[i] = ct[i] - at[i];
    }
}

```

```

    }
    // Waiting time
    for(int i = 0; i < 5; i++)
    {
        wt[i] = tat[i] - bt[i];

        avg_wt = avg_wt + wt[i];
    }
    printf("Process\tAT\tBT\tCT\tWT\tTAT\n");
    for(int i = 0; i < 5; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i],
wt[i], tat[i]);
    }
    avg_wt = avg_wt / 5;
    printf("\nAverage Waiting Time = %.2f\n",
        avg_wt);
    // Gantt Chart
    printf("\nGantt Chart:\n");
    printf("-----\n");
    printf("|  P1  |  P2  |  P3  |  P4  |  P5  |\n");
    printf("-----\n");
    printf("0      6      10      12      15      16\n");
    return 0;
}

```

Q6Deadlock Handling A printing company has 3 printers and 4 scanner units shared by 4 departments. Detect if the current resource allocation state is safe using the Banker's Algorithm. Define Allocation, Max, and Available matrices for 4 processes and 2 resource types Implement the safety algorithm to find a safe sequence Print "Safe sequence: " or "System is in deadlock" based on result

```

#include <stdio.h>

int main()
{
    int allocation[4][2] =
    {
        {1, 0},
        {2, 1},

```

```

        {1, 1},
        {0, 2}
    };
    int max[4][2] =
    {
        {3, 2},
        {2, 2},
        {2, 2},
        {1, 3}
    };
    int available[2] = {2, 1};
    int need[4][2];
    int finish[4] = {0};
    int safe[4];
    int work[2];
    int count = 0;
    // Calculate Need Matrix
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }
    // Copy available to work
    for(int j = 0; j < 2; j++)
    {
        work[j] = available[j];
    }
    // Safety Algorithm
    while(count < 4)
    {
        int found = 0;
        for(int i = 0; i < 4; i++)
        {
            if(finish[i] == 0)
            {
                int possible = 1;
                for(int j = 0; j < 2; j++)
                {
                    if(need[i][j] > work[j])
                    {
                        possible = 0;

```



```

        break;
    }
}
if(possible)
{
    for(int j = 0; j < 2; j++)
    {
        work[j] = work[j] + allocation[i][j];
    }
    safe[count] = i;
    finish[i] = 1;
    found = 1;
    count++;
}
}
}
if(found == 0)
{
    break;
}
}
// Final Result
if(count == 4)
{
    printf("Safe sequence: ");
    for(int i = 0; i < 4; i++)
    {
        printf("P%d ",
            safe[i]);
    }
    printf("\n");
}
else
{
    printf("System is in deadlock\n");
}
return 0;
}

```

Q7 Shared Memory A weather station wants to share real-time temperature readings between a sensor process (writer) and a display process (reader) without using files. Write a producer program that creates shared memory using `shmget()` and writes 5 temperature values. Write a consumer program that attaches to the same segment and reads the values. Detach and destroy shared memory after use with `shmdt()` and `shmctl()`.

writer.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main()
{
    int shmid;
    int *temp;
    int values[5] =
    {30, 32, 31, 33, 35};
    // Create shared memory
    shmid = shmget(1234, 5 * sizeof(int), IPC_CREAT | 0666);
    // Attach shared memory
    temp = (int*) shmat(shmid, NULL, 0);
    // Write values
    for(int i = 0; i < 5; i++)
    {
        temp[i] = values[i];
        printf("Writer wrote: %d\n", temp[i]);
    }
    // Detach shared memory
    shmdt(temp);
    return 0;
}
```

reader.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main()
```

```

{
    int shmid;
    int *temp;
    // Access existing shared memory
    shmid = shmget(1234, 5 * sizeof(int), 0666);
    // Attach shared memory
    temp = (int*) shmat(shmid, NULL, 0);
    // Read values
    for(int i = 0; i < 5; i++)
    {
        printf("Reader read: %d\n",
               temp[i]);
    }
    // Detach shared memory
    shmdt(temp);
    // Destroy shared memory
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}

```

Q8|IPC Classical Problems A bakery has 1 baker (producer) and 3 customers (consumers). The shelf holds at most 5 items. Solve this Producer-Consumer problem using semaphores to prevent race conditions. Use `sem_init()` to create empty, full, and mutex semaphores. Producer adds items using `sem_wait(empty)` → critical section → `sem_post(full)`. Consumers consume items similarly; run with pthread threads and verify no item is consumed twice.

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define SIZE 5
int buffer[SIZE];
int in = 0;
int out = 0;
sem_t empty;
sem_t full;
sem_t mutex;
// Producer Function
void* producer(void* arg)
{
    for(int item = 1; item <= 10; item++)

```

```

    {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = item;
        printf("Producer produced item %d at %d\n", item, in);
        in = (in + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

// Consumer Function
void* consumer(void* arg)
{
    int id = *(int*)arg;
    for(int i = 1; i <= 3; i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumer %d consumed item %d from %d\n", id, item,
out);
        out = (out + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_t p;
    pthread_t c1, c2, c3;
    int id1 = 1;
    int id2 = 2;
    int id3 = 3;
    // Initialize semaphores
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    // Create producer

```

```

pthread_create(&p, NULL, producer, NULL);
// Create consumers
pthread_create(&c1, NULL, consumer, &id1);
pthread_create(&c2, NULL, consumer, &id2);
pthread_create(&c3, NULL, consumer, &id3);
// Wait for completion
pthread_join(p, NULL);
pthread_join(c1, NULL);
pthread_join(c2, NULL);
pthread_join(c3, NULL);
// Destroy semaphores
sem_destroy(&empty);
sem_destroy(&full);
sem_destroy(&mutex);
return 0;
}

```

Q9Disk Scheduling A hospital records system has disk requests at tracks: 98, 183, 37, 122, 14, 124, 65, 67. The disk head starts at track 53. Find total head movement using SSTF scheduling. Implement SSTF in C: at each step, service the closest unvisited request. Print the order in which tracks are serviced. Calculate and print total head movement (sum of absolute differences)

```

#include <stdio.h>
#include <stdlib.h>
int main()
{
    int request[8] = {98, 183, 37, 122, 14, 124, 65, 67};
    int visited[8] = {0};
    int head = 53;
    int total = 0;
    printf("Service Order:\n");
    printf("%d", head);
    for(int i = 0; i < 8; i++)
    {
        int min = 9999;
        int index = -1;
        for(int j = 0; j < 8; j++)
        {
            if(visited[j] == 0)
            {
                int distance =

```

```

        abs(head - request[j]);
        if(distance < min)
        {
            min = distance;
            index = j;
        }
    }
    visited[index] = 1;
    total = total + min;
    head = request[index];
    printf(" -> %d", head);
}
printf("\n\nTotal Head Movement = %d\n", total);
return 0;
}

```

Q10Memory Address Translation A virtual memory system uses paging with page size 4KB. Given a virtual address, compute the physical address using a simple page table. Define a page table array mapping virtual page numbers to frame numbers (e.g., page 0→frame 3, page 1→frame 1, page 2→frame 4) For virtual addresses 0x1A3F and 0x2B10, extract page number and offset, then compute physical address Print both virtual and computed physical addresses in hex

```

#include <stdio.h>
int main()
{
    unsigned int page_table[3] = {3, 1, 4};
    unsigned int virtual_address[2] = {0x1A3F, 0x2B10};
    unsigned int page_size = 4096;
    for(int i = 0; i < 2; i++)
    {
        unsigned int va = virtual_address[i];
        unsigned int page_number = va / page_size;
        unsigned int offset = va % page_size;
        unsigned int frame_number = page_table[page_number];
        unsigned int physical_address = (frame_number * page_size) +
offset;
        printf("\nVirtual Address = 0x%X\n", va);
        printf("Page Number = %u\n", page_number);
        printf("Offset = 0x%X\n", offset);
        printf("Frame Number = %u\n", frame_number);
    }
}

```

```
    printf("Physical Address = 0x%X\n", physical_address);  
}  
return 0;  
}
```